

修正



Thought for 38s >

了解しました。

Decision Console / Portfolio を **世界最高峰寄りの実用仕様**として、Claude にそのまま渡せるレベルでまとめます。

CQOx v2 仕様書 (Claude向け)

対象 : Decision Console / Portfolio

0. ゴールと非ゴール

ゴール

- マーケティングレベルの意思決定を **1 画面で完結**できる Decision Console。
- ポリシー集合に対してポートフォリオ最適化 (利益 vs リスク vs 品質) を行う Portfolio。
- 空の状態 / ポリシー**1本** / 複数本の全ケースで意味のある **UI** を提供。
- Policy Lab で定義したポリシーだけを材料にする。
「アップロードしただけで謎の A~Q ポリシーが並ぶ」挙動は廃止。

非ゴール

- 自動で「勝手に」ポリシーを生成すること (将来のレコメンド機能に分離)。
- マーケティング別の細かい KPI ダッシュボード (これは Grafana / 別画面で扱う)。

1. ドメインモデル (バックエンド共通)

1.1 エンティティ

Dataset

```
id: string
name: string
rows: int
cols: int
```

AnalysisRun

```
id: string
dataset_id: string
treatment_col: string
outcome_col: string
estimators: [string] # ["DR", "IPW", ...]
started_at: datetime
completed_at: datetime | null
status: "running" | "completed" | "failed"
metrics: AnalysisMetrics
```

AnalysisMetrics

```
delta_yen: float           # 推定インクリメンタル利益
delta_yen_ci_lower: float
delta_yen_ci_upper: float
cas_score: float           # Causal Assurance Score 0-1
risk_prob_loss: float      # 損失発生確率 (0-1)
cvar_10: float             # 下位10%の平均損失 (>=0)
roi: float                 # Δ¥ / cost
reach_users: int | null
```

Policy

```
id: string
name: string
description: string
analysis_id: string        # どの因果分析結果に基づくか
```

```
dataset_id: string
channel: string          # "email", "app_push", "multi-channel" など
target_segment_name: string # "LTV>100k & freq>3" 等 (空なら "All customers")
target_segment_sql: string  # WHERE 句 (実行は別システム)
estimated_cost: float      # 施策コスト (円)
status: "draft" | "completed"
```

```
PolicyEvaluation
policy_id: string
delta_yen: float
cas_score: float
risk_prob_loss: float
cvar_10: float
roi: float
reach_users: int | null
verdict: "go" | "canary" | "hold"
```

```
PortfolioFlag
policy_id: string
include: bool          # ポートフォリオ最適化に含めるか
test_only: bool        # 将来のテスト用 (v2では未使用)
```

```
PortfolioOptimizationResult
id: string
time_window: {start_date, end_date}
included_policy_ids: [string]
expected_delta_yen: float
portfolio_cas: float
portfolio_risk: float    # 損失確率のポートフォリオ集約
portfolio_cvar_10: float
portfolio_roi: float
diagnostics: {...}      # 選択理由の説明テキストなど
```

2. Decision Console - Marketing Decisions 仕様

2.1 画面の役割

- 期間を指定して、**「実施候補ポリシーのポートフォリオ全体のパフォーマンス」**を俯瞰する。
- 細粒度の診断や最適化は **Portfolio 画面** に譲り、Decision Console は経営層・マーケティング責任者向けの「サマリー」。

2.2 表示データのスコープ

- 対象となる Policy は：
 - `status == "completed"`
 - `PortfolioFlag.include == true`
- 期間フィルタ：
 - `Last 14 days`, `Last 28 days`, `Last 56 days`
 - 選択した期間内に **evaluation 時刻が含まれる PolicyEvaluation** のみ集計。

2.3 コンポーネント構成

(1) ヘッダ

- タイトル: `Decision Console - マーケティング施策意思決定`
- サブタイトル: `{start_date}~{end_date}` のマーケティング施策ポートフォリオ
- 右側に期間切り替えボタン: `Last 14d | Last 28d | Last 56d`

(2) KPI カード (4枚)

1. Total Incremental Profit

- 値: 期間内の `sum(delta_yen)` (include==true のポリシーのみ)
- サブテキスト: `合計` + 日付範囲

2. Average $\Delta\yen$ / Policy

- 値: `mean(delta_yen)` (ポリシー数>0 のとき)

- サブテキスト: `Target: >= 0`

3. Mean CAS

- 値: `mean(cas_score)`
- サブテキスト: `Target >= 0.75`

4. CVaR (worst 10%)

- 値: `portfolio_cvar_10` (後述の集計アルゴリズム)
- サブテキスト: `Lower is better`

(3) チャート

1. Δ¥ Trend (週次)

- x軸: 週 (ISO week)
- y軸: `sum(delta_yen)` by week
- データが 1 点以下なら、
 - 棒 1本 + 「まだ 1 週間分のデータしかありません」メッセージ。

2. Segment Portfolio

- x軸: セグメント名 or "All customers"
- y軸: そのセグメントに属するポリシーの `sum(delta_yen)`
- データがなければ「セグメント付きポリシーは未定義です」と表示。

3. Channel Performance

- x軸: チャンネル
- 左 y軸: `sum(delta_yen)`
- 右 y軸: `mean(roi)`
- 棒 + ラインの二軸グラフ。

(4) Decision Cards テーブル

- 行: `PortfolioFlag.include == true` かつ期間内のポリシー。
- 列:
 - Policy
 - Dataset
 - Channel
 - Segment
 - Δ¥ (¥フォーマット)
 - ROI (`xx.x x`)
 - CAS
 - Risk (%表記)
 - Verdict (Go / Canary / Hold)
 - Period (日付)
- フィルタ:
 - Verdict タブ: `All | Go | Canary | Hold`
 - テキスト検索: ポリシー名・チャンネル・セグメントに対して fuzzy search。

(5) Empty State (ポリシー0本)

- KPI カードは 0 / N/A を表示。
- チャート部分はグレーのプレースホルダ + 以下のテキスト:

まだポートフォリオ対象のポリシーがありません。

「Policy Lab」で施策を作成し、「Portfolio」で INCLUDE に設定すると、この画面に集計されます。

3. Portfolio - Marketing Portfolio & ROI 仕様

3.0 役割

- Policy Lab で定義完了したポリシー群を材料に、
 - どのポリシーをセットで出すか (INCLUDE/EXCLUDE)
 - その組み合わせの利益・リスク・CAS を最適化する。
- アナリスト〜マネージャー向けの「オペレーション決定画面」。

3.1 上部 : Recommended Portfolio Strategy カード

入力

- PortfolioFlag.include == true のポリシー集合 P
- 各ポリシーの delta_yen, cas_score, risk_prob_loss, cvar_10, roi, estimated_cost

最適化ロジック (シンプル版)

- 制約 :
 - CAS $\geq$ θ_{cas} (例: 0.15, 将来 0.75 に上げる)
 - Risk $\leq$ θ_{risk} (例: 0.6)
- 目的 :
 - sum(delta_yen) 最大
- 実装案 :
 - v1 は 単純に「include == true の全ポリシー」を採用し、
 - 制約を満たしていない場合は **WARN** としてカード上に表示。
 - 将来 v2 で Integer Programming や Greedy で最適サブセットを選ぶ。

表示内容

- Expected $\Delta\yen$: sum(delta_yen) で $\yen$ フォーマット。
- CAS Score : weighted_mean(cas_score, weight=delta_yen>0 ? delta_yen : 1)
- Risk Score : weighted_mean(risk_prob_loss, weight=abs(delta_yen) + 1)
- ROI : sum(delta_yen) / sum(estimated_cost) (cost>0 のとき)
- テキストボックス (Decision Rationale) 例 :

選択された {n} 件のポリシーが、CAS $\geq$ { θ_{cas} } & Risk $\leq$ { θ_{risk} } の制約下で最も高い $\Delta\yen$ を提供します。このポートフォリオは Pareto 効率的です。

- もし制約を満たさない場合 :

現在の INCLUDE 状態では、CAS または Risk が目標を満たしていません。

- CAS 平均 : {cas} (目標 { θ_{cas} })
- Risk 平均 : {risk} (目標 { θ_{risk} })

「Portfolio Policies」でポリシーを調整してください。

3.2 中段 : Pareto Frontier & Portfolio Contribution

(1) Pareto Frontier (Profit vs Risk)

- x軸 : Risk (0~1)
- y軸 : Profit ($\Delta\yen$)
- 各ポリシーを点としてプロット。
- CAS に応じて色分け :
 - CAS $\geq$ 0.8: High (緑)
 - 0.6-0.8: Medium (黄)
 - < 0.6: Low (赤)
- INCLUDE==true のポリシーを塗りつぶし、EXCLUDE を半透明に。
- Frontier 線 :
 - INCLUDE==true のポリシーのみから Pareto 前線を計算し、破線で表示。
- データ点が 1 or 0 の時 :
 - 点1つ + メッセージ :

ポリシー数が 1 件のため、Pareto Frontier は参考値としてのみ表示しています。
複数のポリシーを定義して比較することで、トレードオフが明確になります。

(2) Portfolio Contribution

- 推奨ポートフォリオに含まれるポリシーのみを対象。
- 上位5件まで棒グラフで表示 :
 - y軸 : $\Delta\yen$ (単位 : 円)
 - テキスト : {rank}. {policy_name} - $\yen$ {delta_yen} ({share}% of total)

3.3 下段 : Portfolio Policies テーブル

行の条件

- `Policy.status == "completed"` のみ。
- **Dataset** から自動生成されるポリシーはない。
すべて Policy Lab 由来。

列

- Policy Name
- Dataset
- Channel
- Segment
- Δ¥
- ROI
- Risk
- CAS
- Verdict (Go/Canary/Hold)
- Include トグル (ボタン or スイッチ)

振る舞い

- Include トグル :
 - ON → `PortfolioFlag.include = true`
 - OFF → `PortfolioFlag.include = false`
 - トグル変更時に上部の KPI, Frontier, Contribution を再フェッチ。
- テーブル上部フィルタ :
 - タブ : `All | Include | Test | Exclude`
 - テキスト検索 : `policy name / dataset / channel / segment`。

4. Portfolio の状態ごとの UX 仕様

4.1 Empty State (ポリシー 0 本)

条件

- `Policy.status == "completed"` のレコードが 1 つもない。

UI

1. 上部カード :
 - すべて 0 / N/A 表示。
 - Rationale ボックスに以下の文 :

まだ評価済みのマーケティング施策ポリシーがありません。

 1. 「Causal Design」で因果推論を実行し、
 2. 「Policy Lab」でポリシーを作成して保存すると、
ここにポートフォリオ候補が表示されます。
2. Pareto Frontier / Contribution :
 - グレー背景 + アイコン + 説明テキストのみ。
3. Portfolio Policies テーブル :
 - 行は 0。
 - テーブル中央に :

No policies to display.

「Policy Lab」で施策を作成してください。

4.2 Policy 1本だけ存在する場合

条件

- `completed` な Policy が 1 本。
- `include==true` かはユーザ次第。

UI ルール

1. Include==false の場合：

- Recommended Portfolio：
 - Expected Δ¥ = ¥0
 - カード下部に：

現在、ポートフォリオに含まれているポリシーはありません。

下のテーブルで少なくとも 1 件のポリシーを *INCLUDE* に設定してください。

2. Include==true の場合：

- Recommended Portfolio：
 - Expected Δ¥ = そのポリシーの Δ¥
 - CAS / Risk / ROI はそのまま。
- Pareto Frontier：
 - 1 点だけをプロットし、前述の 1 点用メッセージを表示。
- Contribution：
 - そのポリシー 1 本の棒グラフのみ。share=100%。

3. テーブル：

- 1 行のみ。Include の ON/OFF によって上部が即座に変化。

5. API & バックエンド契約 (Claude用)

5.1 Decision Console

```
GET /api/v1/console/summary?window=28d
```

Response

```
{
  "window": { "start": "2025-10-29", "end": "2025-11-25" },
  "kpis": {
    "total_delta_yen": 2397000.0,
    "avg_delta_yen": 133000.0,
    "mean_cas": 0.15,
    "portfolio_cvar_10": 0.0
  },
  "trend": [
    { "week": "2025-W48", "delta_yen": 2200000.0 }
  ],
  "segment_portfolio": [
    { "segment": "High LTV", "delta_yen": 1800000.0 },
    { "segment": "All customers", "delta_yen": 597000.0 }
  ],
  "channel_performance": [
    { "channel": "email", "delta_yen": 2000000.0, "roi": 4.5 },
    { "channel": "app_push", "delta_yen": 397000.0, "roi": 2.1 }
  ],
  "decision_cards": [
    {
      "policy_id": "pol-001",
      "policy_name": "Email Campaign Q4",
      "dataset": "marketing_campaign_10k_processed",
      "channel": "email",
      "segment": "High LTV",
      "delta_yen": 133000.0,
      "roi": 5.0,
      "cas": 0.15,
      "risk": 0.592,
      "verdict": "go",
      "period": "2025-11-25"
    }
  ]
}
```

5.2 Portfolio

(1) Summary + Optimization

```
GET /api/v1/portfolio/summary?window=28d
```

Response

```
{
  "window": { "start": "2025-10-29", "end": "2025-11-25" },
  "constraints": {
    "min_cas": 0.15,
    "max_risk": 0.6
  },
  "recommended": {
    "included_policy_ids": ["pol-005", "pol-010", "pol-012", "pol-013", "pol-017"],
    "expected_delta_yen": 665883.0,
    "portfolio_cas": 0.15,
    "portfolio_risk": 0.59,
    "portfolio_cvar_10": 0.0,
    "portfolio_roi": 5.0,
    "diagnostics": {
      "status": "ok | constraint_violation",
      "message": "選択された 5 件のポリシーが、CAS >= 0.15 & Risk <= 0.59 の制約下で最も高い Δ¥ を提供します。"
    }
  },
  "pareto_points": [
    {
      "policy_id": "pol-005",
      "policy_name": "Policy E",
      "delta_yen": 133000.0,
      "risk": 0.59,
      "cas": 0.15,
      "on_frontier": true
    }
  ],
  "contribution": [
    {
      "policy_id": "pol-005",
      "policy_name": "Policy E",
      "delta_yen": 133000.0,
      "share": 0.2
    }
  ]
}
```

(2) Policies テーブル

```
GET /api/v1/portfolio/policies?window=28d
```

Response

```
{
  "policies": [
    {
      "policy_id": "pol-001",
      "name": "Policy A",
      "dataset": "marketing_campaign_10k_processed",
      "channel": "Multi-channel",
      "segment": "All customers",
      "delta_yen": 133000.0,
      "roi": 5.0,
      "risk": 0.592,
      "cas": 0.15,
      "verdict": "go",
      "include": false
    },
    {
      "policy_id": "pol-005",
      "name": "Policy E",

```

```

    "dataset": "marketing_campaign_10k_processed",
    "channel": "Multi-channel",
    "segment": "All customers",
    "delta_yen": 133000.0,
    "roi": 5.0,
    "risk": 0.592,
    "cas": 0.15,
    "verdict": "go",
    "include": true
  }
]
}

```

(3) INCLUDE トグル更新

POST /api/v1/portfolio/policies/{policy_id}/include
Content-Type: application/json

```
{ "include": true }
```

Response: 200 OK, body { "ok": true }

6. 実装メモ (Claude向け)

6.1 フロント (React + Vite + TanStack Query)

- ページコンポーネント :
 - pages/DecisionConsolePage.tsx
 - pages/PortfolioPage.tsx
- データフェッチ hooks 例 :

```

// useDecisionConsole.ts
export function useDecisionConsole(windowDays: 14 | 28 | 56) {
  return useQuery({
    queryKey: ["console", windowDays],
    queryFn: () => client.get(`/api/v1/console/summary?window=${windowDays}`)
  });
}

```

```

// usePortfolio.ts
export function usePortfolio(windowDays: 14 | 28 | 56) {
  const summary = useQuery(...);
  const policies = useQuery(...);
  return { summary, policies };
}

```

- INCLUDE 更新用 mutation :

```

export function useToggleInclude() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ policyId, include }: { policyId: string; include: boolean }) =>
      client.post(`/api/v1/portfolio/policies/${policyId}/include`, { include }),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["portfolio"] });
      queryClient.invalidateQueries({ queryKey: ["console"] });
    },
  });
}

```

6.2 バックエンド (FastAPI / Python 想定)

- エンドポイント定義は上記 API を参照。
- Portfolio サマリ計算は純粋関数として実装するとテストしやすい。

7. What this does (要約)

- **ゴール**
Decision Console と Portfolio を、「ポリシー集合の利益・リスク・因果品質」を一貫して扱える世界水準の意思決定画面として定義し直す。
- **入力**
 - Causal Design が出す AnalysisRun + Metrics
 - Policy Lab が定義する Policy + PolicyEvaluation
 - PortfolioFlag (INCLUDE/EXCLUDE)
- **出力 / 副作用**
 - Decision Console : 期間内のポートフォリオのサマリーKPI と Decision Cards 一覧。
 - Portfolio : 推奨ポートフォリオ、Pareto Frontier、寄与度ランキング、INCLUDE 操作。
 - 副作用は **INCLUDE フラグ更新のみ**。分析結果そのものは変更しない。
- **なぜこのアプローチか**
 - 「データセットから謎のポリシーが湧く」状態を排除し、
Causal → Policy Lab → Portfolio → Decision Console の **一方向パイプライン**を明確化。
 - Empty / 1本 / 複数本の全ケースに対して UI の意味が崩れないため、
大企業での PoC〜本番利用でも誤解が起きにくい。
- **実行・検証方法 (例)**

```
# バックエンドテスト
pytest tests/test_portfolio_aggregation.py -q
pytest tests/test_console_summary.py -q

# フロント開発
cd frontend
npm run dev

# E2E (Playwright など)
npx playwright test portfolio.spec.ts
```

8. Expert insight (Google/Meta/NASA level)

「勝手に増えるオブジェクト」を **UI から排除せよ**。

メトリクスやポリシーが「どこから来たか」がユーザに明示されないと、
どれだけ高度なアルゴリズムを積んでも **信頼性の上限が低く抑えられます**。

Causal Design → Policy Lab → Portfolio → Decision Console という **明示的なライフサイクル**を持たせることで、
監査性・説明責任・プロダクト寿命のすべてが一段上がります。